CASE STUDY

Enterprise Data Governance & Data Quality Transformation

Platform Snowflake + Databricks
Tooling dbt · Great Expectations · Snowflake Tasks · Slack Alerting
Duration 14 weeks (Discovery → Stabilization)
Role Data Quality Lead · Data Governance Architect

1. Executive Summary

A product engineering team was generating high-volume operational data consumed by executive dashboards, financial reporting pipelines, and downstream ML workloads running across Snowflake and Databricks. Despite heavy daily usage, the platform had no schema standards, no validation controls, and no defined ownership — producing data that was systematically unreliable and broadly distrusted by the business.

This case study documents the design and delivery of a comprehensive Data Governance and Data Quality framework that transformed the platform over 14 weeks. The engagement spanned root cause analysis, standards authorship, automated validation implementation, a weighted Data Quality Scoring model, real-time monitoring, and RACI-based ownership formalization.

Measured Outcomes — 14-week engagement
‣ Duplicate records reduced from 12.4 % of ingestion volume to 0.3 %
‣ Critical field completeness improved from 71 % average to 99.1 %
‣ BI manual validation effort cut from ~18 hours / week to < 2 hours
‣ Platform Data Quality Score (DQS) raised from 54 / 100 to 91 / 100
‣ Critical data incidents: 6 per month pre-engagement → 0 in 90 days post-launch ‣ Cross-platform (Snowflake / Databricks) record variance: from ±4.1 % to < 0.01 %

2. Business Context

The application produced transactional records spanning order management, customer interactions, and financial events. This data was consumed across three distinct and business-critical use cases:

- Executive and operational KPI dashboards refreshed on a 2-hour SLA
- Financial and regulatory reporting pipelines with reconciliation requirements
- Downstream ML feature stores and analytics pipelines running in Databricks

Data moved through a three-layer architecture: external stage ingestion into raw Snowflake schemas, transformation through a staging layer, and promotion to a curated Delta Lake layer in Databricks, with selected datasets reverse-synced into Snowflake for BI consumption. At each boundary, data quality degraded — not because the architecture was flawed, but because no quality gates existed at any layer transition.

3. Problem Statement & Measured Baseline

A structured two-week assessment established quantified baselines before any remediation work began. Each issue category was measured independently to establish a defensible starting point and to prioritize the remediation backlog.

Metric	Before	After
Duplicate records	12.4 % of raw ingestion	**0.3 % (98 % reduction)**
Critical field completeness	71 % average	**99.1 % average**
Column naming variants (8 tables)	23 distinct variants	**1 enforced standard**
Invalid order_amount records	3.7 % negative or zero	**< 0.05 %**
Average data freshness vs. 2-hr SLA	6.2 hours	**1.4 hours**
Datasets with a named Data Owner	0 of 14	**14 of 14**
Weekly BI manual validation effort	~18 hours	**< 2 hours**
Platform DQS	54 / 100	**91 / 100**

4. Root Cause Analysis

Root causes were categorized across four dimensions to map each problem to its correct remediation track. Misclassifying a governance problem as a technical one — or vice versa — would produce the wrong fix.

Standards Void

No naming conventions existed or were enforced. The same concept appeared under multiple identifiers across tables ingested by the same pipeline. At assessment time, the customer identifier alone appeared as: cust_id, custID, customer_id, CUST_KEY, and c_id — five variants across eight tables in a single domain. Joins built on assumptions about column names failed silently, producing incorrect aggregations rather than explicit errors.

Shift-Right Validation

All data quality checks were performed manually by BI analysts after data had reached the curated layer. The cost of detecting a defect post-curation was estimated internally at 40× the cost of catching it at ingestion. This reactive model was also non-scalable: as data volumes grew, manual validation effort grew proportionally, not sub linearly.

Ownership Vacuum

No Data Owner, Steward, or Custodian roles were defined. In practice, data quality was described as a shared responsibility — a framing that, in the absence of explicit accountability, means no responsibility.

Issues were filed as application bugs against the engineering team, who had no governance mandate and no framework for prioritizing data quality fixes against feature delivery.

Tooling Used Only for Storage

Snowflake Tasks, Streams, and the Snowflake data quality extension were unused. Databricks Delta Lake table constraints and expectations were unconfigured. Dbt was partially adopted for transformations but had zero tests defined across all models. Great Expectations were on the roadmap but unimplemented. The platform held enterprise-grade tooling that was delivering commodity storage.

5. Implementation Phases

The engagement was structured in four sequential phases. Each phase had a defined gate: the outputs of one phase became the inputs to the next. No technical implementation began until governance design was complete and signed off by Data Owners.

Phase	Weeks	Key Deliverables
Phase 1 — Discovery & Baseline	1 – 2	Data profiling, root cause classification, stakeholder interviews, baseline DQS per dataset, remediation backlog prioritization
Phase 2 — Governance Design	3 – 5	Naming & coding standards, RACI model, DQ dimension definitions, business rule catalogue, SLA contracts per dataset, CI/CD enforcement spec
Phase 3 — Technical Implementation	6 – 11	dbt tests, Great Expectations suites, Snowflake Task scheduling, cross-platform reconciliation job, DQS calculation pipeline, Slack alerting integration
Phase 4 — Stabilization & Handover	12 – 14	Monitoring dashboards, runbooks per dataset, team training sessions, ownership transfer sign-offs, 90-day SLA review cadence established

6. Governance Implementation

6.1 Naming & Coding Standards

Standards were defined in a living specification document and enforced at the pull request stage via a custom sqlfluff ruleset integrated into the CI/CD pipeline. Violations failed the build. The standard was not advisory — it was a deployment gate.

Object Type	Convention	Example
Table	domain_entity_layer	sales_orders_curated
Column	snake_case, no abbreviations	order_created_ts

Primary key	<entity>_id	order_id, customer_id
Timestamp	<event>_ts	order_created_ts, record_updated_ts
Boolean flag	is_<condition>	is_active, is_deleted, is_reconciled
Surrogate key	<entity>_sk	customer_sk
Foreign key	<entity>_id (same as PK ref)	customer_id in orders table

6.2 Ownership Model — RACI

The RACI model was formalized before any technical work began and recorded in the data catalog. All 14 production datasets were assigned named individuals in each role. The model was reviewed and re-confirmed at the Phase 4 handover.

Role	Assigned To	Accountability
Data Owner	Business domain VP or Director	Approves SLA definitions, signs off DQ thresholds, holds escalation authority for SLA breaches
Data Steward	Domain subject-matter expert	Defines and validates business rules, approves DQS threshold changes, reviews anomalies
Data Custodian	Data Engineering team member	Implements validation rules, maintains pipelines, resolves technical incidents within SLA
Data Consumer	BI / Analytics / ML team leads	Reports anomalies via defined channel, participates in SLA reviews, owns downstream contracts

6.3 SLA Contracts

Each dataset received a formal SLA contract defining freshness expectations, completeness thresholds, and escalation procedures. SLAs were agreed between the Data Owner and Data Consumer before being codified as monitoring thresholds. Breach conditions triggered automated alerts rather than relying on manual discovery.

SLA Dimension	Threshold	Breach Action
Freshness	Max 2 hours since last record	Slack alert → Data Custodian within 15 min
Completeness	>= 98 % on critical mandatory fields	DQS drop alert → Data Steward review
Uniqueness	0 duplicate natural keys in curated	Pipeline halt → incident raised
DQS score	>= 80 / 100 daily	< 80: Custodian alert | < 65: Owner escalation + pipeline hold

7. Data Quality Framework

Five quality dimensions were formally defined, each with a business rule catalogue, SQL-based measurement query, and a weight within the DQS composite score. Dimensions were not chosen arbitrarily — they map directly to the failure modes identified in the root cause analysis.

7.1 Uniqueness

Business rule: each natural key must appear exactly once in the curated layer. Deduplication in the staging-to-curated transition uses a ROW_NUMBER partition strategy with explicit tie-breaking on ingestion timestamp. The selection logic is transparent and auditable.

```
-- Deduplication: retain the latest ingestion per natural key
WITH ranked_orders AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY ingested_ts DESC
    ) AS rn
  FROM staging.orders
)
SELECT * EXCLUDE(rn)
FROM   ranked_orders
WHERE  rn = 1.

-- Post-load check: assert zero duplicates survive to curated
SELECT order_id, COUNT(*) AS cnt
FROM   curated.orders
GROUP  BY order_id
HAVING cnt > 1;
```

7.2 Completeness

Business rule: order_id, customer_id, order_amount, and order_created_ts are mandatory. A NULL in any of these fields is a completeness violation recorded against the daily DQS calculation. Per-field percentages are logged for trend analysis.

```
-- Field-level completeness audit with percentage scoring
SELECT
  COUNT(*)                                                  AS total_records,
  ROUND(COUNT(order_id)          / COUNT(*) * 100, 2)     AS pct_order_id,
  ROUND(COUNT(customer_id)       / COUNT(*) * 100, 2)     AS pct_customer_id,
  ROUND(COUNT(order_amount)      / COUNT(*) * 100, 2)     AS pct_order_amount,
  ROUND(COUNT(order_created_ts) / COUNT(*) * 100, 2)     AS pct_created_ts
FROM curated.orders;
```

7.3 Validity

Business rules: order_amount must be greater than zero; order_status must belong to the approved domain set; order_created_ts must not be a future timestamp. The validity check returns a labelled flag per failing record, enabling targeted remediation rather than bulk rejection.

```
-- Multi-rule validity classification
SELECT
  order_id,
  order_amount,
  order_status,
  order_created_ts,
  CASE
    WHEN order_amount <= 0
      THEN 'INVALID_AMOUNT'
    WHEN order_status NOT IN (
           'pending','confirmed','shipped','delivered','cancelled')
      THEN 'INVALID_STATUS'
    WHEN order_created_ts > CURRENT_TIMESTAMP()
      THEN 'FUTURE_TIMESTAMP'
    ELSE 'VALID'
  END AS validity_flag
FROM curated.orders
WHERE order_amount <= 0
   OR order_status NOT IN (
        'pending','confirmed','shipped','delivered','cancelled')
   OR order_created_ts > CURRENT_TIMESTAMP();
```

7.4 Consistency — Cross-Platform Reconciliation

With data processed in parallel across Snowflake and Databricks, a daily reconciliation job confirmed that record counts and financial aggregates aligned within a tolerance of ±0.01 %. Divergence above that threshold triggered an automatic pipeline hold pending investigation.

```
-- Cross-platform reconciliation  (scheduled via Snowflake Task, daily 06:00 UTC)
SELECT
  'snowflake'                    AS platform,
  COUNT(*)                       AS record_count,
  COUNT(DISTINCT order_id)       AS unique_orders,
  ROUND(SUM(order_amount), 2)    AS total_revenue
FROM snowflake_curated.orders
UNION ALL
SELECT
  'databricks',
  COUNT(*),
  COUNT(DISTINCT order_id),
  ROUND(SUM(order_amount), 2)
FROM databricks_curated.orders;
```

7.5 Timeliness

Freshness was measured against the 2-hour SLA agreed with Data Owners. A Snowflake Task running every 30 minutes evaluated the gap between the most recent record timestamp and the current time, returning a structured status that fed directly into the alerting pipeline.

```
-- Freshness SLA evaluation  (runs every 30 min via Snowflake Task)
SELECT
  MAX(order_created_ts)                                        AS last_record_ts,
  CURRENT_TIMESTAMP()                                          AS checked_at,
  TIMESTAMPDIFF('minute',
    MAX(order_created_ts), CURRENT_TIMESTAMP())                AS lag_minutes,
  CASE
    WHEN TIMESTAMPDIFF('hour',
         MAX(order_created_ts), CURRENT_TIMESTAMP()) > 4
      THEN 'CRITICAL_BREACH'
    WHEN TIMESTAMPDIFF('hour',
         MAX(order_created_ts), CURRENT_TIMESTAMP()) > 2
      THEN 'SLA_BREACH'
    ELSE 'OK'
  END AS freshness_status
FROM curated.orders;
```

8. Automated Validation Strategy

Validation was implemented at three independent layers — dbt at model promotion, Great Expectations as a post-load contract, and Snowflake Tasks for continuous monitoring. Each layer catches a different failure mode. A defect escaping one layer is caught by the next.

Validation Layer	Trigger	Failure Behaviour
dbt schema + custom tests	Every PR merge and scheduled run	Test failure blocks model promotion to curated layer
Great Expectations suite	Post-load, after each curated refresh	Suite failure raises incident and suspends downstream pipelines
Snowflake Task monitoring	Every 30 minutes, continuous	Threshold breach triggers Slack alert to on-call Custodian

8.1 dbt — Schema and Business Rule Tests

All curated models were equipped with schema.yml definitions covering generic tests, relationship integrity, accepted values, and custom SQL assertions. Source freshness configuration was added to detect upstream ingestion failures independently of downstream model runs.

```
# models/curated/schema.yml
version: 2

sources:
  - name: staging
    freshness:
      warn_after:  { count: 2, period: hour }
      error_after: { count: 4, period: hour }
    tables:
```

```
      - name: orders
        loaded_at_field: ingested_ts

models:
  - name: orders
    description: 'Curated orders — deduplicated, validated, SLA-monitored'
    columns:
      - name: order_id
        tests: [ not_null, unique ]
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to:    ref('customers')
              field: customer_id
      - name: order_amount
        tests:
          - not_null
          - dbt_utils.expression_is_true:
              expression: '> 0'
      - name: order_status
        tests:
          - accepted_values:
              values: [ 'pending','confirmed','shipped','delivered','cancelled' ]
      - name: order_created_ts
        tests:
          - not_null
          - dbt_utils.expression_is_true:
              expression: '<= current_timestamp()'
```

8.2 Great Expectations — Curated Layer Contract

A Great Expectations expectation suite was deployed as an independent post-load contract on the curated orders table. The suite runs after every curated refresh and is version-controlled alongside the dbt models. Failure of any expectation raises an incident and suspends promotion to the BI layer.

```
# expectations/suites/curated_orders_suite.py

validator.expect_column_values_to_not_be_null('order_id')
validator.expect_column_values_to_be_unique('order_id')

validator.expect_column_values_to_not_be_null('customer_id')
validator.expect_column_values_to_not_be_null('order_amount')

validator.expect_column_values_to_be_in_set(
    column='order_status',
    value_set=['pending','confirmed','shipped','delivered','cancelled']
)
validator.expect_column_values_to_be_between(
    column='order_amount',
    min_value=0.01,
    max_value=1_000_000
)
validator.expect_table_row_count_to_be_between(
    min_value=10_000,    # volume floor — anomaly detection
    max_value=500_000    # volume ceiling — anomaly detection
)
```

```
validator.expect_column_max_to_be_between(
    column='order_created_ts',
    max_value='now'      # no future timestamps in curated layer
)
```

9. Data Quality Scoring Model

The DQS is a weighted composite score calculated daily per dataset. It converts multiple quality dimensions into a single number that is visible to Data Owners, Stewards, and downstream consumers. The score drives threshold-based alerting: a DQS below 80 triggers a Slack alert to the Data Custodian; a score below 65 triggers escalation to the Data Owner and suspends pipeline promotion to the BI layer.

Dimension	Weight	Rationale
Completeness	30 %	Mandatory fields — highest business consequence when missing
Uniqueness	25 %	Duplicate records directly corrupt KPI aggregations
Validity	20 %	Business rule violations produce incorrect downstream results
Consistency	15 %	Cross-platform divergence breaks ML feature stores
Timeliness	10 %	SLA freshness — critical but partially external-dependency-driven

```
-- Daily DQS calculation  (Snowflake Task, 07:00 UTC)
WITH completeness AS (
  SELECT
    ROUND(
      ( COUNT(order_id) + COUNT(customer_id)
      + COUNT(order_amount) + COUNT(order_created_ts) )
      / NULLIF(COUNT(*) * 4, 0) * 100, 2
    ) AS score
  FROM curated.orders
),
uniqueness AS (
  SELECT
    ROUND(
      (1 - COUNT(d.order_id) * 1.0 / NULLIF(COUNT(*), 0)) * 100, 2
    ) AS score
  FROM curated.orders o
  LEFT JOIN (
    SELECT order_id FROM curated.orders
    GROUP BY order_id HAVING COUNT(*) > 1
  ) d ON o.order_id = d.order_id
),
validity AS (
  SELECT
    ROUND(
      SUM(CASE WHEN order_amount > 0
               AND order_status IN (
                 'pending','confirmed','shipped','delivered','cancelled')
               AND order_created_ts <= CURRENT_TIMESTAMP()
               THEN 1 ELSE 0 END)
      * 1.0 / NULLIF(COUNT(*), 0) * 100, 2
    ) AS score
```

```
  FROM curated.orders
)
SELECT
  CURRENT_DATE()                                    AS score_date,
  c.score                                           AS completeness_score,
  u.score                                           AS uniqueness_score,
  v.score                                           AS validity_score,
  ROUND(
    c.score * 0.30 +
    u.score * 0.25 +
    v.score * 0.20,
  2)                                                AS dqs_composite
FROM completeness c, uniqueness u, validity v;
```

10. Results & Business Impact

10.1 Quantitative Outcomes

All metrics were measured against the baselines established in Phase 1. Post-implementation measurements were taken at 30, 60, and 90 days after go-live and averaged. The 90-day window was chosen to exclude the stabilization period and reflect steady-state platform behavior.

Metric	Before	After
Duplicate records (% of ingestion)	12.4 %	**0.3 % — 98 % reduction**
Critical field completeness	71.0 % avg.	**99.1 % avg.**
Weekly BI manual validation effort	~18 hrs / week	**< 2 hrs / week — 89 % reduction**
Platform DQS	54 / 100	**91 / 100**
Critical data incidents / month	6	**0 in 90-day post-launch window**
Cross-platform record variance	± 4.1 %	**< 0.01 %**
Avg. data freshness vs. 2-hr SLA	6.2 hours	**1.4 hours**
Datasets with named Data Owner	0 of 14	**14 of 14**

10.2 Operational Outcomes

- Validation pipeline fully automated — zero manual checks required before BI dashboard refresh.
- Snowflake Tasks execute DQS calculations on a 30-minute cadence with threshold-based Slack alerting.
- dbt CI/CD integration gates every pull request — schema and data quality test failures block deployment.
- Great Expectations suites version-controlled and co-located with dbt models — treated as first-class code.

- RACI model embedded in the data product intake process — new datasets require Owner and Steward assignment before entering the ingestion pipeline.

10.3 Business Outcomes

- BI stakeholder trust survey moved from 2.1 / 5 to 4.6 / 5 at the 90-day post-launch review.
- Finance team decommissioned its external reconciliation spreadsheet — a manual artefact maintained weekly for two years.
- ML engineering team adopted curated orders as a trusted feature source, replacing a bespoke pre-processing job they had maintained independently.
- Data quality score now presented in the monthly data platform review attended by domain VPs — governance is visible at the executive level.

11. Key Learnings

- **Design quality in — do not inspect it out.** Shifting validation left to the ingestion layer eliminated the most expensive category of defects. The cost of catching a NULL at ingestion is negligible; the cost of discovering it in a board-level KPI is not. Every downstream fix applied to a defect that could have been blocked at source represents recoverable waste.

- **Governance without enforcement is documentation.** Naming standards became real only when violations blocked deployment. The CI/CD sqlfluff integration was the single highest-leverage governance control in the entire framework. Without it, the standards document would have been read once and ignored.

- **Ownership must precede technology.** The RACI model was established and signed off before any technical implementation began. Teams will not maintain what they do not own. Deploying monitoring before ownership is defined creates alerts with no one accountable for responding to them.

- **Scoring drives accountability.** The DQS model transformed data quality from a qualitative complaint into a measurable metric visible to Data Owners. Once scores were reported at the monthly executive review, governance adoption accelerated without additional mandate — visibility created its own incentive.

- **Cross-platform consistency is its own failure mode.** With Snowflake and Databricks as co-processing platforms, data divergence between environments created a category of defects that neither platform team detected independently. Teams working within a single platform have no natural visibility into cross-system drift. The reconciliation job filled a blind spot that would otherwise have persisted indefinitely.

12. Conclusion

This engagement demonstrates that data quality is an engineering discipline, not a remediation exercise. A platform that was generating systematically unreliable data was not suffering from a technology deficiency — it was suffering from an absence of standards, ownership, and enforcement. The tools to build a high-quality data platform were already present. What was missing was the governance architecture to use them purposefully.

Within 14 weeks, the platform moved from a DQS of 54 to 91, eliminated its backlog of critical incidents, and restored the confidence of stakeholders who had been manually compensating for data failures for years. The governance model is intentionally portable: the RACI structure, DQ dimension framework, and SLA contract template are now applied to every new dataset entering the platform — meaning governance is applied before a dataset breaks production, not after.

Framework Components Delivered
‣ Naming & Coding Standards — enforced via CI/CD sqlfluff linting on every PR
‣ Data Governance RACI — named Owner, Steward, and Custodian for all 14 datasets
‣ 5-Dimension DQ Framework — business rules + SQL measurement per dimension
‣ dbt Validation Layer — schema.yml tests, relationship tests, source freshness ‣ Great Expectations Suites — post-load contracts on all curated models
‣ Weighted DQS Model — daily composite score with dual-threshold alerting
‣ Cross-Platform Reconciliation Job — Snowflake vs. Databricks, daily
‣ Monitoring & Alerting — Snowflake Tasks + Slack, 30-minute cadence
‣ Data Catalog Entries — lineage, definitions, and SLA docs for all datasets
‣ Runbooks & Handover Documentation — per-dataset, owned by Data Custodians